

Implement an End to End CNN Model for Image Classification Task.

✓ Task 1: Data Understanding and Visualization:

1. Load and visualize images from a dataset stored in directories, where each subdirectory represents a class. You are expected to write a code:

- Get the list of class directories from the train folder.
- Select one image randomly from each class.
- Display the images in a grid format with two rows using matplotlib.

```
import os
import random
import matplotlib.pyplot as plt
from PIL import Image

# Define the path to the training dataset
train_dir = '/content/drive/MyDrive/Artificial Intelligence/Week5/FruitinAmazon/train' # Replace with the actual path if different

# Get the list of class directories
class_dirs = [d for d in os.listdir(train_dir) if os.path.isdir(os.path.join(train_dir, d))]

# Select one random image from each class
selected_images = []
class_labels = []

for class_dir in class_dirs:
    class_path = os.path.join(train_dir, class_dir)
    images = [f for f in os.listdir(class_path) if f.endswith((''.png', '.jpg', '.jpeg'))]

    if images:
        random_image = random.choice(images)
        selected_images.append(os.path.join(class_path, random_image))
        class_labels.append(class_dir)

# Determine grid size (2 rows)
num_classes = len(selected_images)
cols = (num_classes + 1) // 2 # Ensure two rows
rows = 2

# Create figure and axes
fig, axes = plt.subplots(rows, cols, figsize=(12, 6))

for idx, (image_path, label) in enumerate(zip(selected_images, class_labels)):
    img = Image.open(image_path)
    ax = axes[idx // cols, idx % cols]
    ax.imshow(img)
    ax.set_title(label)
    ax.axis("off")

# Remove empty subplots if any
for idx in range(len(selected_images), rows * cols):
    fig.delaxes(axes[idx // cols, idx % cols])

plt.tight_layout()
plt.show()
```



graviola



acai



tucuma



cupuacu



pupunha



guarana



• What did you Observe?

When visualizing the images, i observed that each class has a distinct image, confirming that the dataset is well-structured. Some images appear larger or smaller, lighting conditions diffre across images, and fruits appear against various backgrounds. These variations mean that our image recognition model will need to be flexible enough to identify fruits correctly regardless of these differences.

2. Check for Corrupted Image: Write a script that verifies whether the image in the train directory are valid. If any corrupted images are found, the script must remove the image from the directory and print the message which image have been removed, if none found print "No Corrupted Images Found."

```
import os
from PIL import Image

# Define the path to the training dataset
train_dir = '/content/drive/MyDrive/Artificial Intelligence/Week5/FruitinAmazon/train' # Replace with the actual path if different

# List all class directories
class_dirs = [d for d in os.listdir(train_dir) if os.path.isdir(os.path.join(train_dir, d))]

corrupted_images = [] # List to store corrupted image paths

# Iterate through each class subdirectory
for class_dir in class_dirs:
    class_path = os.path.join(train_dir, class_dir)

    # List all image files in the directory
    images = [f for f in os.listdir(class_path) if f.lower().endswith(('.png', '.jpg', '.jpeg'))]

    for image in images:
        image_path = os.path.join(class_path, image)

        try:
            # Attempt to open and verify the image
            with Image.open(image_path) as img:
                img.verify() # Verify the image integrity

        except (IOError, SyntaxError):
            # If an error occurs, consider it a corrupted image
            corrupted_images.append(image_path)
            os.remove(image_path)
            print(f"Removed corrupted image: {image_path}")
```

```
# Final report
if not corrupted_images:
    print("No Corrupted Images Found.")
```

➡ No Corrupted Images Found.

✓ Task 2: Loading and Preprocessing Image Data in keras:

```
import tensorflow as tf
# Define image size and batch size
img_height = 128 # Example image height
img_width = 128 # Example image width
batch_size = 32
validation_split = 0.2 # 80% training, 20% validation
# Create a preprocessing layer for normalization
rescale = tf.keras.layers.Rescaling(1./255) # Normalize pixel values to [0, 1]
# Create training dataset with normalization
train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    train_dir,
    labels="inferred",
    label_mode="int",
    image_size=(img_height, img_width),
    interpolation="nearest",
    batch_size=batch_size,
    shuffle=True,
    validation_split=validation_split,
    subset="training",
    seed=123
)
# Apply the normalization (Rescaling) to the dataset
train_ds = train_ds.map(lambda x, y: (rescale(x), y))
# Create validation dataset with normalization
val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    train_dir,
    labels="inferred",
    label_mode="int",
    image_size=(img_height, img_width),
    interpolation="nearest",
    batch_size=batch_size,
    shuffle=False,
    validation_split=validation_split,
    subset="validation",
    seed=123
)
# Apply the normalization (Rescaling) to the validation dataset
val_ds = val_ds.map(lambda x, y: (rescale(x), y))
```

➡ Found 90 files belonging to 6 classes.
Using 72 files for training.
Found 90 files belonging to 6 classes.
Using 18 files for validation.

✓ Task 3 - Implement a CNN with

Follow the following Structure and Hyper - parameters:

Convolutional Architecture:

- Convolutional Layer 1:
 - Filter Size (F): (3, 3)
 - Number of Filters (k): 32
 - Padding (P): same
 - Stride (s): 1
- Activation Layer: ReLU activation
- Pooling Layer 1: Max pooling
 - Filter Size (F): (2, 2)

- Stride (s): 2
- Convolutional Layer 2:
 - Filter Size (F): (3, 3)
 - Number of Filters (k): 32
 - Padding (P): same
 - Stride (s): 1
- Activation Layer: ReLU activation
- Pooling Layer 2: Max pooling
 - Filter Size (F): (2, 2)
 - Stride (s): 2

Fully Connected Network Architecture:

- Flatten Layer: Flatten the input coming from the convolutional layers
- Input Layer:
- Hidden Layer - 2:
 - Number of Neurons: 64
 - Number of Neurons: 128
- Output Layer:
 - Number of Neurons: num classes (number of output classes)

```
import numpy as np

# If using tf.data.Dataset
unique_labels = set()
for _, labels in train_ds:
    unique_labels.update(np.unique(labels.numpy()))
print("Unique labels:", unique_labels)
print("Number of classes:", len(unique_labels))

➡ Unique labels: {np.int32(0), np.int32(1), np.int32(2), np.int32(3), np.int32(4), np.int32(5)}
   Number of classes: 6

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Function to create the CNN model
def create_cnn_model(input_shape, num_classes):
    model = keras.Sequential([
        # Convolutional Layer 1
        layers.Conv2D(filters=32, kernel_size=(3, 3), padding="same", strides=1, activation="relu", input_shape=input_shape),
        layers.MaxPooling2D(pool_size=(2, 2), strides=2),

        # Convolutional Layer 2
        layers.Conv2D(filters=32, kernel_size=(3, 3), padding="same", strides=1, activation="relu"),
        layers.MaxPooling2D(pool_size=(2, 2), strides=2),

        # Flatten layer
        layers.Flatten(),

        # Fully Connected Layers
        layers.Dense(64, activation="relu"),
        layers.Dense(128, activation="relu"),

        # Output Layer (Softmax for multi-class classification)
        layers.Dense(num_classes, activation="softmax")
    ])

    return model

# Define parameters
input_shape = (128, 128, 3) # Example input shape (128x128 RGB images)
num_classes = 6 # Adjust based on dataset
```

```
# Create the model
model = create_cnn_model(input_shape, num_classes)

# Print model summary
model.summary()
```

🔗 /usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input\_shape` /`input\_shape\_tuple` argument to the `__init__` method of `Conv2D`. It is deprecated. It will be removed in Keras 3.0.0. Please use `input_shape=(None, None, None, None)` instead.

Model: "sequential_5"

Layer (type)	Output Shape	Param #
conv2d_10 (Conv2D)	(None, 128, 128, 32)	896
max_pooling2d_10 (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_11 (Conv2D)	(None, 64, 64, 32)	9,248
max_pooling2d_11 (MaxPooling2D)	(None, 32, 32, 32)	0
flatten_5 (Flatten)	(None, 32768)	0
dense_15 (Dense)	(None, 64)	2,097,216
dense_16 (Dense)	(None, 128)	8,320
dense_17 (Dense)	(None, 6)	774

Total params: 2,116,454 (8.07 MB)
Trainable params: 2,116,454 (8.07 MB)
Non-trainable params: 0 (0.00 B)

✓ Task 4: Compile the Model

Model Compilation

- Choose an appropriate optimizer (e.g., Adam), loss function (e.g., sparse categorical crossentropy), and evaluation metric (e.g., accuracy).

```
# Compile the model
model.compile(
    optimizer="adam", # Adam optimizer
    loss="sparse_categorical_crossentropy", # Loss function for multi-class classification
    metrics=["accuracy"] # Evaluate model performance
)

print("Model compiled successfully!")
```

🔗 Model compiled successfully!

✓ Task 4: Train the Model

Model Training

- Use the `model.fit()` function to train the model. Set the batch size to 16 and the number of epochs to 250.
- Use `val_ds` for validation.
- Use callbacks such as `ModelCheckpoint` and `EarlyStopping` for saving the best model and avoiding overfitting.

```
# Callbacks
checkpoint = ModelCheckpoint(
    "best_model.h5", # Save the best model based on validation accuracy
    monitor="val_accuracy", # Monitor validation accuracy
    save_best_only=True, # Save only the best model
    mode="max", # We want to maximize validation accuracy
    verbose=1
)

early_stopping = EarlyStopping(
    monitor="val_loss", # Monitor validation loss
    patience=10, # Stop training if no improvement for 10 epochs
    restore_best_weights=True, # Restore the best model weights after stopping
    verbose=1
)

# Combine callbacks
```

```
# combine callbacks
callbacks = [checkpoint, early_stopping]

# Train the model
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=250,
    batch_size=16,
    callbacks=callbacks
)

3/3 ----- 0s 311ms/step - accuracy: 1.0000 - loss: 0.0988
Epoch 9: val_accuracy did not improve from 0.88889
3/3 ----- 2s 389ms/step - accuracy: 1.0000 - loss: 0.0987 - val_accuracy: 0.8333 - val_loss: 0.5100
Epoch 10/250
3/3 ----- 0s 328ms/step - accuracy: 1.0000 - loss: 0.0472
Epoch 10: val_accuracy did not improve from 0.88889
3/3 ----- 3s 509ms/step - accuracy: 1.0000 - loss: 0.0479 - val_accuracy: 0.8333 - val_loss: 0.7027
Epoch 11/250
3/3 ----- 0s 451ms/step - accuracy: 0.9902 - loss: 0.0592
Epoch 11: val_accuracy did not improve from 0.88889
3/3 ----- 2s 651ms/step - accuracy: 0.9891 - loss: 0.0594 - val_accuracy: 0.8889 - val_loss: 0.5417
Epoch 12/250
3/3 ----- 0s 755ms/step - accuracy: 1.0000 - loss: 0.0258
Epoch 12: val_accuracy did not improve from 0.88889
3/3 ----- 4s 1s/step - accuracy: 1.0000 - loss: 0.0275 - val_accuracy: 0.8889 - val_loss: 0.3189
Epoch 13/250
3/3 ----- 0s 317ms/step - accuracy: 0.9954 - loss: 0.0249
Epoch 13: val_accuracy did not improve from 0.88889
3/3 ----- 3s 412ms/step - accuracy: 0.9931 - loss: 0.0302 - val_accuracy: 0.8889 - val_loss: 0.3459
Epoch 14/250
3/3 ----- 0s 311ms/step - accuracy: 0.9803 - loss: 0.1077
Epoch 14: val_accuracy did not improve from 0.88889
3/3 ----- 3s 477ms/step - accuracy: 0.9783 - loss: 0.1142 - val_accuracy: 0.8889 - val_loss: 0.9458
Epoch 15/250
3/3 ----- 0s 332ms/step - accuracy: 0.9190 - loss: 0.3705
Epoch 15: val_accuracy did not improve from 0.88889
3/3 ----- 2s 418ms/step - accuracy: 0.9253 - loss: 0.3441 - val_accuracy: 0.8333 - val_loss: 0.9906
Epoch 16/250
3/3 ----- 0s 311ms/step - accuracy: 0.9549 - loss: 0.1606
Epoch 16: val_accuracy did not improve from 0.88889
3/3 ----- 1s 391ms/step - accuracy: 0.9557 - loss: 0.1595 - val_accuracy: 0.8333 - val_loss: 0.4676
Epoch 17/250
3/3 ----- 0s 754ms/step - accuracy: 1.0000 - loss: 0.0090
Epoch 17: val_accuracy did not improve from 0.88889
3/3 ----- 4s 880ms/step - accuracy: 1.0000 - loss: 0.0088 - val_accuracy: 0.8889 - val_loss: 0.4404
Epoch 18/250
3/3 ----- 0s 314ms/step - accuracy: 0.9954 - loss: 0.0551
Epoch 18: val_accuracy did not improve from 0.88889
3/3 ----- 3s 400ms/step - accuracy: 0.9931 - loss: 0.0596 - val_accuracy: 0.8889 - val_loss: 0.4830
Epoch 19/250
3/3 ----- 0s 316ms/step - accuracy: 1.0000 - loss: 0.0572
Epoch 19: val_accuracy did not improve from 0.88889
3/3 ----- 2s 482ms/step - accuracy: 1.0000 - loss: 0.0550 - val_accuracy: 0.8889 - val_loss: 0.4237
Epoch 20/250
3/3 ----- 0s 314ms/step - accuracy: 0.9902 - loss: 0.0383
Epoch 20: val_accuracy did not improve from 0.88889
3/3 ----- 1s 403ms/step - accuracy: 0.9891 - loss: 0.0397 - val_accuracy: 0.8889 - val_loss: 0.4044
Epoch 21/250
3/3 ----- 0s 457ms/step - accuracy: 1.0000 - loss: 0.0480
Epoch 21: val_accuracy did not improve from 0.88889
3/3 ----- 3s 608ms/step - accuracy: 1.0000 - loss: 0.0462 - val_accuracy: 0.8889 - val_loss: 0.3913
Epoch 22/250
3/3 ----- 0s 807ms/step - accuracy: 1.0000 - loss: 0.0185
Epoch 22: val_accuracy did not improve from 0.88889
3/3 ----- 3s 1s/step - accuracy: 1.0000 - loss: 0.0186 - val_accuracy: 0.8889 - val_loss: 0.4109
Epoch 22: early stopping
Restoring model weights from the end of the best epoch: 12.
```

Task 5: Evaluate the Model

Model Evaluation

- After training, evaluate the model using `model.evaluate()` on the test set to check the test accuracy and loss.

```
# Evaluate the model on the test dataset
test_ds = tf.keras.preprocessing.image_dataset_from_directory(
    '/content/drive/MyDrive/Artificial Intelligence/Week5/FruitinAmazon/test', # specify the path to your test directory
    batch_size=16,
    shuffle=True,
    seed=12345,
    validation_split=0.2,
    image_size=(224, 224),
    include_subfolders=False
)
```

```

        labels= interred ,
        label_mode='int',
        image_size=(128, 128),
        batch_size=batch_size
    )

test_loss, test_accuracy = model.evaluate(test_ds)
print(f"Test Loss: {test_loss}")
print(f"Test Accuracy: {test_accuracy}")

```

 Found 30 files belonging to 6 classes.
 1/1 _____ 7s 7s/step - accuracy: 0.7000 - loss: 121.1894
 Test Loss: 121.1893539428711
 Test Accuracy: 0.699999988079071

✓ Task 6: Save and Load the Model

Model Saving and Loading

- Save the trained model to an .h5 file using `model.save()`.
- Load the saved model and re-evaluate its performance on the test set.

```

model.save('final_model.h5')

```

 WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is consi

```

loaded_model = tf.keras.models.load_model('final_model.h5')

```

 WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you t

```

# Evaluate the loaded model
test_loss, test_accuracy = loaded_model.evaluate(test_ds)

print(f"Test Accuracy: {test_accuracy:.4f}")

# Re-evaluate the loaded model on the test set
test_loss, test_accuracy = loaded_model.evaluate(test_ds)
print(f"Loaded Model - Test Loss: {test_loss}")
print(f"Loaded Model - Test Accuracy: {test_accuracy}")

```

 1/1 _____ 1s 931ms/step - accuracy: 0.7667 - loss: 0.9131
 Loaded Model - Test Loss: 0.9130719304084778
 Loaded Model - Test Accuracy: 0.7666666507720947

✓ Task 7: Predictions and Classification Report

Making Predictions

- Use `model.predict()` to make predictions on test images.
- Convert the model's predicted probabilities to digit labels using `np.argmax()`.
- Also use `from sklearn.metrics import classification_report` to report the Classification Report of your Model Performance.

```

import numpy as np
from sklearn.metrics import classification_report

# Make predictions on the test dataset
predictions = model.predict(test_ds)

# Convert predicted probabilities to class labels
predicted_labels = np.argmax(predictions, axis=1)

# Get the true labels from the test dataset
true_labels = []
for _, labels in test_ds:
    true_labels.extend(labels.numpy())

# Convert the true labels into a numpy array
true_labels = np.array(true_labels)

```

```
# Print the classification report
print(classification_report(true_labels, predicted_labels))
```

```
1/1 ----- 0s 303ms/step
```

	precision	recall	f1-score	support
0	0.20	0.20	0.20	5
1	0.33	0.40	0.36	5
2	0.14	0.20	0.17	5
3	0.00	0.00	0.00	5
4	0.17	0.20	0.18	5
5	0.50	0.20	0.29	5
accuracy			0.20	30
macro avg	0.22	0.20	0.20	30
weighted avg	0.22	0.20	0.20	30

```
import os
```

```
# List files in the current directory
print(os.listdir())
```

```
[ '.config', 'drive', 'fruit_classifier.h5', 'fruit_classifier_model.h5', 'final_model.h5', 'best_model.h5', 'sample_data' ]
```

```
import shutil
```

```
# Move the model to a new directory
shutil.move("final_model.h5", "/content/drive/MyDrive/Artificial Intelligence/Week5")
```

```
['/content/drive/MyDrive/Artificial Intelligence/Week5/final_model.h5']
```